

Le problème initial

On part d'un nombre entier N supérieur à 1. S'il est pair, on le divise par 2, sinon on le multiplie par 3 puis on ajoute 1. Puis on recommence avec le résultat et ainsi de suite ... on s'arrête dès qu'on arrive à 1.

Par exemple : on part du nombre 10.

- 10 est pair donc on le divise par 2 : on obtient 5.
- 5 est impair donc on le multiplie par 3 puis on ajoute 1 : on obtient 16.
- 16 est pair donc on le divise par 2 : on obtient 8.
- 8 est pair donc on le divise par 2 : on obtient 4.
- 4 est pair donc on le divise par 2 : on obtient 2.
- 2 est pair donc on le divise par 2 : on obtient 1.

On a obtenu 1 : On arrête.

En partant du nombre 10, on a donc obtenu successivement 5, 16, 8, 4, 2 puis 1, donc six étapes pour arriver à 1 et le plus grand nombre rencontré est 16.

Autre exemple : si on part de 7, on obtient successivement 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2 puis 1, donc 16 étapes pour arriver à 1 et le plus grand nombre rencontré est 52.



Le but du sujet est de déterminer le plus petit entier de départ N nécessitant 30 étapes pour arriver à 1 en précisant le plus grand nombre alors rencontré.

Partie A (sur table) (30 minutes)

1) Compléter l'écriture de la fonction **suivant**(*n*) ci-dessous afin qu'elle renvoie le nombre suivant *n*. Par exemple, on aura **suivant**(10)=5 et **suivant**(1)=1.

```
Fonction suivant (n: type ..... ) # type du résultat : .....  
Début  
  Si n== ..... Alors  
    Retourner(.....)  
  SinonSi n%2== .....  
    Retourner(.....)  
  Sinon  
    Retourner(.....)  
  FinSi  
FinFonction
```

2) Quel est le rôle de l'algorithme suivant ?

```
Variables : n (entier), ch (chaîne)  
Début  
  n ← 10  
  ch ← " # chaîne vide  
  TantQue suivant(n)!=1 Faire  
    ch = ch + suivant(n) + '-' # concaténation  
    n ← suivant(n)  
  FinTantQue  
  Afficher ch+'1'  
Fin
```

3) Écrire une fonction **trajet**(*n*) qui retourne la liste de tous les nombres rencontrés lorsqu'on part de l'entier *n*. Par exemple on devra avoir **trajet**(10)=[5,16,8,4,2,1].

4) Écrire une fonction **max**(*n*) qui à un nombre de départ *n*, retourne le plus grand nombre alors rencontré.

5) Écrire un algorithme déterminant puis affichant le plus petit entier *N* nécessitant 30 étapes pour arriver à 1 ainsi que le plus grand nombre alors rencontré.

Partie B (sur machine) (30 minutes)

1) Implémenter l'algorithme de la partie A en corrigeant les éventuelles erreurs.

2) Exécuter le programme puis noter les résultats obtenus.

FICHE REPONSE
A rendre après les 30 premières minutes

Fiche réponse

Nom de l'algorithme

Rôle :

Entrée(s) :

Sortie(s) :

Variable(s) globale(s) et type(s) :

..... Fonctions

Fonction suivant(n: type) # type du résultat :

Début

Si n== **Alors**

Retourner(.....)

Si non Si n%2==

Retourner(.....)

Si non

Retourner(.....)

FinSi

FinFonction

Fonction trajet(n: type) # type du résultat :

Variable(s) locale(s) et type(s) :

Début

FinFonction

Fonction max(n: type) # type du résultat :

Variable(s) locale(s) et type(s) :

Début

FinFonction

.....Partie principale

Début

Fin

Résultats obtenus après exécution :

Rôle de l'algorithme de la question 2 :